

Senior Full Stack Engineer (AI Native)

Technical Vetting Assignment

Instructions & Requirements

Take-Home Assignment: Multi-Tenant Pulse Surveys (Focused Slice)

Context

The platform you will be working on helps organizations drive behaviour adoption by gathering lightweight weekly signals from their people and turning them into simple insights. Build a minimal, production-quality slice of a multi-tenant SaaS that demonstrates senior-level judgment across backend and data, with a very small React UI to exercise key flows. Timebox your work and prioritize a clean, coherent vertical slice over breadth.

Requirements

Build a minimal multi-tenant pulse survey capability with the following constraints:

- **You must use an AI coding tool** (Claude Code, Codex, Cursor, GitHub Copilot or similar) to complete this assignment. This role is AI-native, and how you direct and validate AI output is assessed alongside what you build. Completing the assignment without an AI tool will count against you.
- Core technologies: TypeScript, NestJS, PostgreSQL, React.
- Run locally without external identity providers or paid cloud services.
- Users belong to exactly one organization and have one of two roles: Manager, Member.
- Data from one organization must not be accessible to another.
- Keep scope tight; prioritize the smallest end-to-end "happy path" first.
- In SOLUTION.md, note trade-offs, known gaps, and next steps you would take with more time.

Flexibility to reduce decision churn:

- Either a rolling 7-day window or a calendar week is acceptable for "this week" — state your choice.
- Any isolation approach is acceptable for this slice (for example, application-layer scoping or database row policies) — state your choice and why.

Task 1: Multi-Tenant Pulse Surveys (Backend)

Create a backend service that supports weekly pulse surveys per organization.

Business requirements:

- Organizations can have pulse surveys with up to three questions to keep scope small.
- Supported question types for this slice: rating from 1–5, and yes/no.
- Managers can create and manage surveys for their own organization.
- Members can submit one response per active week for a given survey.
- Provide a 7-day (or weekly) summary for a survey that includes:
 - Overall completion count and completion rate (relative to the number of members in that organization)
 - Per-question rollups:
 - For rating: an average and a count
 - For yes/no: counts per option
- Ensure strict data isolation across organizations.
- Provide seed data for at least two organizations and a few users per role to demonstrate isolation.

Notes:

- Choose how to represent users, roles, organizations, surveys, and responses.
- Choose a local-friendly way to identify the current user and organization (for example, a simple token, session, or header).
- You may create surveys via seed/fixture or a simple admin endpoint/CLI to save UI time.

Task 2: Minimal React App (Essential Flows)

Build a small React app to exercise the backend:

- As a Member:
 - View your organization's active survey.
 - Submit your response.
- As a Manager:
 - View the 7-day (or weekly) summary for a survey.

Demo expectations:

- Include at least two organizations in your demo flow to clearly show that data is isolated.
- Keep the UI minimal and focused on the flows above. A simple way to "log in" locally is acceptable (for example, selecting a seeded user).

Task 3 (Design-Only): Production Readiness on AWS

In SOLUTION.md, include a brief design note (no code required) covering:

- How you would deploy this on AWS (for example, container service for the app, managed PostgreSQL, static hosting for the frontend).
- Your approach to storing and serving an organization logo image in production, with the business goal of minimizing backend bandwidth and cost while keeping access secure.
- The tenancy, security, and scaling considerations you would prioritize first.

Task 4: AI-Assisted Delivery

Show how you worked with your AI tools, not just what they produced.

- **Workflow.** In SOLUTION.md, explain your AI workflow end to end: which tools you used, how you set up the task for them, how you broke the work down, what you delegated to the AI and what you kept for yourself, how you reviewed and iterated on the output, and what you would change next time.
- **Agent instructions.** Commit the instructions file you wrote for your tool (for example CLAUDE.md, AGENTS.md, .cursorrules or equivalent). It should reflect the constraints of this project.
- **Spec before code.** Commit a short spec or plan before the implementation commits. Keep it brief; the point is that it exists and that the code follows it.
- **Validation.** As part of the workflow explanation, describe how you validated the AI's output: what you checked, what you rejected or rewrote, and how you knew the result was correct.
- **Session logs.** Export or copy your AI session transcripts into an ai-logs/ folder in the repository. Redact anything personal. If your tool cannot export transcripts, say so in SOLUTION.md and describe the sessions instead.

Deliverables

1. A single GitHub repository containing:

- All the source code for the assignment
- A README.md with instructions to run locally.
-

A SOLUTION.md describing the design and trade-offs you made, the AWS design note, and your AI workflow including how you validated AI output.

- An ai-logs/ folder with your AI session transcripts.
- A commit history that shows how the work progressed. Do not squash to a single commit.

2. Video Demo (5-10 minutes using Loom or similar):

- Demonstrate the working project by doing a voice over
- Walk through your code architecture
- Explain key design decisions and any trade-offs made
- Explain your AI workflow and show it in action, including at least one place where you corrected or rejected the output

Time Estimate

This should take ± 4 hours to complete, but you can take as long as you need.